

语法分析
自顶向下-LL(1)文法

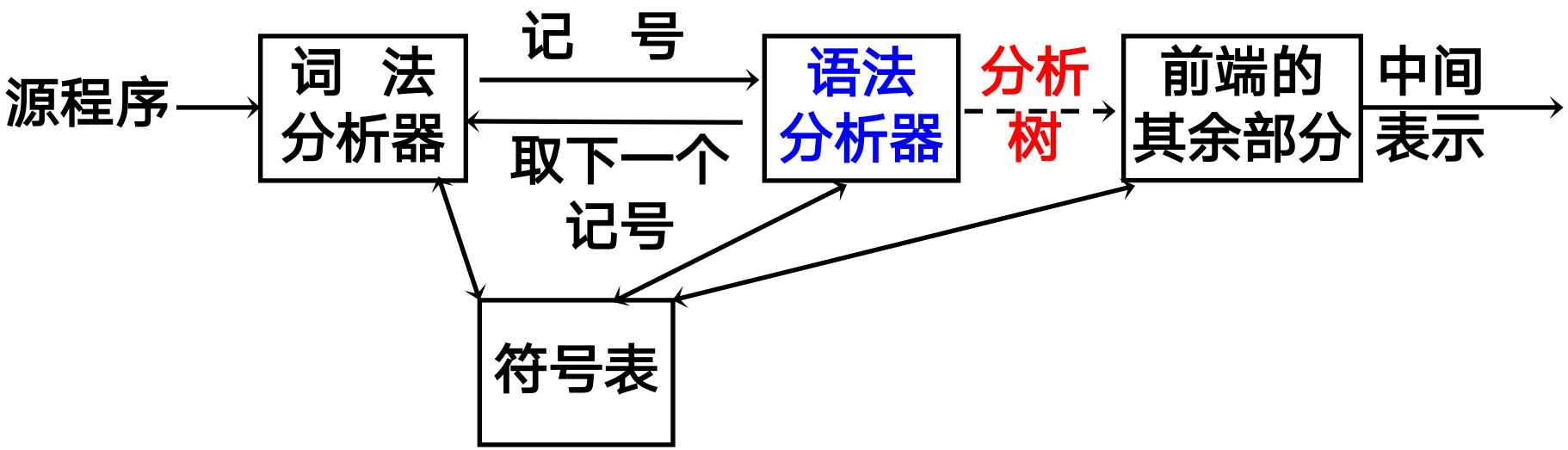
徐 伟

国家高性能计算中心(合肥)、信息与计算机国家级实验教学示范中心

计算机科学与技术学院

2026年09月24日

本节提纲



- 自顶向下分析方法
  - LL(1)文法
  - 非递归预测分析方法

预测分析法（Predictive parsing）

- **与递归下降法相似，但**
  - 不会对若干产生式进行尝试
  - 没有回溯
  - 通过向前看一些记号来预测需要用到的产生式

- **此方法接受LL(k)文法**
  - L-means “left-to-right” scan of input
  - L-means “leftmost derivation”
  - k-means “predict based on k tokens of lookahead”
  - In practice, LL(1) is used

LL(1)文法

- 对文法加什么样的限制可以保证没有回溯？

- 先定义两个和文法有关的函数
  - FIRST(α) = $\{a \mid \alpha \Rightarrow^* a\ldots, a \in V_T\}$

    意义：可从α推导得到的串的首符号的集合

  - FOLLOW(A) = $\{a \mid S \Rightarrow^* \ldots Aa\ldots,\ \ a \in V_T\}$

    意义：可能在推导过程中紧跟在A右边的终结符号的集合

LL(1)文法：FIRST(X）

- **计算FIRST(*X*), $X \in V_T \cup V_N$**
  - $X \in V_T$, FIRST(X) = $\{X\}$
  - $X \in V_N$且 $X \rightarrow \varepsilon$

    则将 ε加入到FIRST(*X*)
  - $X \in V_N$且 $X \rightarrow Y_1\ Y_2\ \ldots Y_k$
    - 如果 $a \in$ FIRST(Y_i)且ε在FIRST(Y_1), …, FIRST(Y_{i-1})中， 则将 a加入到FIRST(X)
    - 如果 ε 在FIRST(Y_1), …, FIRST(Y_k)中， 则将ε 加入到FIRST(X)

FIRST集合只包括终结符和ε

表达式文法：无左递归的

- 例 $E \to TE'$

  $E' \to +TE' \mid \varepsilon$

  $T \to FT'$

  $T' \to *FT' \mid \varepsilon$

  $F \to (E) \mid \mathbf{id}$

$\mathbf{FIRST}(F) = \{ (, \mathbf{id} \}$

- $X \in V_T$, $\mathbf{FIRST}(X) = \{X\}$
- $X \in V_N$且 $X \to \varepsilon$, $\varepsilon \in \mathbf{FIRST}(X)$
- $X \in V_N$且 $X \to Y_1 Y_2 \ldots Y_k$
  - 如果 $a \in \mathbf{FIRST}(Y_i)$且$\varepsilon$在$\mathbf{FIRST}(Y_1), \ldots, \mathbf{FIRST}(Y_{i-1})$中，则$a \in \mathbf{FIRST}(X)$
  - 如果 ε 在$\mathbf{FIRST}(Y_1), \ldots, \mathbf{FIRST}(Y_k)$中，则$\varepsilon \in \mathbf{FIRST}(X)$

表达式文法：无左递归的

- 例 $E \to TE'$

  $E' \to +TE' \mid \varepsilon$

  $T \to FT'$

  $T' \to *FT' \mid \varepsilon$

  $F \to (E) \mid \mathrm{id}$

- $X \in V_T$, FIRST(X) = $\{X\}$
- $X \in V_N$且 $X \to \varepsilon$, $\varepsilon \in$ FIRST(X)
- $X \in V_N$且 $X \to Y_1 Y_2 \ldots Y_k$
  - 如果 $a \in$ FIRST(Y_i)且ε在FIRST(Y_1), …, FIRST(Y_{i-1})中， 则$a \in$ FIRST(X)
  - 如果 ε 在FIRST(Y_1), …, FIRST(Y_k)中，则$\varepsilon \in$ FIRST(X)

FIRST(F) = { (, id } = FIRST(T) = FIRST(E)

表达式文法：无左递归的

- 例 $E \to TE'$

  $E' \to +TE' \mid \varepsilon$

  $T \to FT'$

  $T' \to *FT' \mid \varepsilon$

  $F \to (E) \mid \mathrm{id}$

- $X \in V_T$, FIRST(X) = {X}
- $X \in V_N$且 $X \to \varepsilon$, $\varepsilon \in$ FIRST(X)
- $X \in V_N$且 $X \to Y_1 Y_2 \ldots Y_k$
  - 如果 $a \in$ FIRST(Y_i)且ε在FIRST(Y_1), …, FIRST(Y_{i-1})中， 则$a \in$ FIRST(X)
  - 如果 ε 在FIRST(Y_1), …, FIRST(Y_k)中，则$\varepsilon \in$ FIRST(X)

FIRST(F) = { (, id } = FIRST(T) = FIRST(E)

FIRST(E') = {+, ε}

表达式文法：无左递归的

- 例 $E \to TE'$

  $E' \to +TE' \mid \varepsilon$

  $T \to FT'$

  $T' \to *FT' \mid \varepsilon$

  $F \to (E) \mid \mathrm{id}$

- $X \in V_T$, FIRST(X) = $\{X\}$
- $X \in V_N$且 $X \to \varepsilon$, $\varepsilon \in$ FIRST(X)
- $X \in V_N$且 $X \to Y_1 Y_2 \ldots Y_k$
  - 如果 $a \in$ FIRST(Y_i)且ε在FIRST(Y_1), …, FIRST(Y_{i-1})中， 则$a \in$ FIRST(X)
  - 如果 ε 在FIRST(Y_1), …, FIRST(Y_k)中，则$\varepsilon \in$ FIRST(X)

FIRST(F) = { (, id } = FIRST(T) = FIRST(E)

FIRST(E') = {+, ε}

FRIST(T') = {*, ε}

表达式文法：无左递归的

- 例 $E \to TE'$

  $E' \to +TE' \mid \varepsilon$

  $T \to FT'$

  $T' \to *FT' \mid \varepsilon$

  $F \to (E) \mid \mathrm{id}$

- $X \in V_T$, FIRST(X) = $\{X\}$
- $X \in V_N$且 $X \to \varepsilon$, $\varepsilon \in$ FIRST(X)
- $X \in V_N$且 $X \to Y_1 Y_2 \ldots Y_k$
  - 如果 $a \in$ FIRST(Y_i)且ε在FIRST(Y_1), …, FIRST(Y_{i-1})中， 则$a \in$ FIRST(X)
  - 如果 ε 在FIRST(Y_1), …, FIRST(Y_k)中，则$\varepsilon \in$ FIRST(X)

FIRST(E) = FIRST(T) = FIRST(F) = { (, id }

FIRST(E') = {+, ε}

FRIST(T') = {*, ε}

LL(1)文法：FOLLOW(A）

- 计算FOLLOW(A), $A \in V_N$
  - 当A是开始符号，$加入到FOLLOW($A$)，$是输入串的结束符号
  - 如果$A \rightarrow \alpha B\beta$，则FIRST($\beta$)-{ε}加入到FOLLOW($B$)
  - 如果$A \rightarrow \alpha B$ 或$A \rightarrow \alpha B\beta$且$\varepsilon \in$ FIRST(β)，则FOLLOW(A)加入到FOLLOW(B)

表达式文法：无左递归的

- 例 $E \to TE'$

$E' \to +TE' \mid \varepsilon$

$T \to FT'$

$T' \to *FT' \mid \varepsilon$

$F \to (E) \mid \mathbf{id}$

- 当A是开始符号，$ ∈ FOLLOW(A)
- $A \to \alpha B\beta$，FIRST(β)-{ε} ⊆ FOLLOW(B)
- $A \to \alpha B$ 或 $A \to \alpha B\beta$且ε ∈ FIRST(β)，FOLLOW(A) ⊆ FOLLOW(B)

FIRST(E) = FIRST(T) = FIRST(F) = { (, id }

FIRST(E') = {+, ε}

FRIST(T') = {*, ε}

FOLLOW(E) = {), $}

表达式文法：无左递归的

- 例 $E \to TE'$

$E' \to +TE' \mid \varepsilon$

$T \to FT'$

$T' \to *FT' \mid \varepsilon$

$F \to (E) \mid \mathbf{id}$

- 当A是开始符号，$ ∈ FOLLOW(A)
- $A \to \alpha B\beta$，FIRST(β)-{ε} ⊆ FOLLOW(B)
- $A \to \alpha B$ 或 $A \to \alpha B\beta$且ε ∈ FIRST(β)，FOLLOW(A) ⊆ FOLLOW(B)

FIRST(E) = FIRST(T) = FIRST(F) = { (, id }

FIRST(E') = {+, ε}

FRIST(T') = {*, ε}

FOLLOW(E) = {), $} = FOLLOW($E'$)

表达式文法：无左递归的

- 例 $E \to TE'$

$E' \to +TE' \mid \varepsilon$

$T \to FT'$

$T' \to *FT' \mid \varepsilon$

$F \to (E) \mid \mathrm{id}$

- 当A是开始符号，$ ∈ FOLLOW(A)
- $A \to \alpha B\beta$，FIRST(β)-{ε} ⊆ FOLLOW(B)
- $A \to \alpha B$ 或 $A \to \alpha B\beta$且ε ∈ FIRST(β)，FOLLOW(A) ⊆ FOLLOW(B)

FIRST(E) = FIRST(T) = FIRST(F) = { (, id }

FIRST(E') = {+, ε}

FRIST(T') = {*, ε}

FOLLOW(E) = {), $} = FOLLOW($E'$)

FOLLOW(T) = {+,), $}

表达式文法：无左递归的

- 例 $E \to TE'$

$E' \to +TE' \mid \varepsilon$

$T \to FT'$

$T' \to *FT' \mid \varepsilon$

$F \to (E) \mid \mathbf{id}$

- 当A是开始符号，$ ∈ FOLLOW(A)
- $A \to \alpha B\beta$，FIRST(β)-{ε} ⊆ FOLLOW(B)
- $A \to \alpha B$ 或 $A \to \alpha B\beta$且ε ∈ FIRST(β)，FOLLOW(A) ⊆ FOLLOW(B)

FIRST(E) = FIRST(T) = FIRST(F) = { (, id }

FIRST(E') = {+, ε}

FRIST(T') = {*, ε}

FOLLOW(E) = {), $} = FOLLOW($E'$)

FOLLOW(T) = {+,), $} = FOLLOW ($T'$)

表达式文法：无左递归的

- 例 $E \to TE'$

$E' \to +TE' \mid \varepsilon$

$T \to FT'$

$T' \to *FT' \mid \varepsilon$

$F \to (E) \mid \mathbf{id}$

- 当A是开始符号，$ ∈ FOLLOW(A)
- $A \to \alpha B\beta$，FIRST(β)-{ε} ⊆ FOLLOW(B)
- $A \to \alpha B$ 或 $A \to \alpha B\beta$且ε ∈ FIRST(β)，FOLLOW(A) ⊆ FOLLOW(B)

FIRST(E) = FIRST(T) = FIRST(F) = { (, id }

FIRST(E') = {+, ε}

FRIST(T') = {*, ε}

FOLLOW(E) = {), $} = FOLLOW($E'$)

FOLLOW(T) = {+,), $} = FOLLOW ($T'$)

FOLLOW(F) = {*,+,), $}

LL(1)文法

- **LL(1)文法的定义**

  任何两个产生式$A \to \alpha \mid \beta$都满足下列条件：

  - $\mathrm{FIRST}(\alpha) \cap \mathrm{FIRST}(\beta) = \varnothing$
  - 若$\beta \Rightarrow^* \varepsilon$，那么$\mathrm{FIRST}(\alpha) \cap \mathrm{FOLLOW}(A) = \varnothing$

LL(1)文法

- **LL(1)文法的定义**

  任何两个产生式$A \to \alpha \mid \beta$都满足下列条件：

  - $\mathrm{FIRST}(\alpha) \cap \mathrm{FIRST}(\beta) = \varnothing$
  - 若$\beta \Rightarrow^* \varepsilon$，那么$\mathrm{FIRST}(\alpha) \cap \mathrm{FOLLOW}(A) = \varnothing$

- **该条件存在的必要性**
  - 容易理解
  - 每次通过输入词法单元记号和FIRST集合匹配产生式的时候，需要有唯一的选择

LL(1)文法

- **LL(1)文法的定义**

  任何两个产生式$A \to \alpha \mid \beta$都满足下列条件：

  - $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \varnothing$
  - 若$\beta \Rightarrow^* \varepsilon$，那么$\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \varnothing$

❑假设$\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \{a\}$

$a \in \text{FIRST}(\alpha)$: $A \Rightarrow^* a\alpha'$

$a \in \text{FOLLOW}(A)$: $B \Rightarrow^* \ldots A\, a \ldots$

LL(1)文法

- **LL(1)文法的定义**

  任何两个产生式$A \to \alpha \mid \beta$都满足下列条件：

  - $\mathrm{FIRST}(\alpha) \cap \mathrm{FIRST}(\beta) = \varnothing$
  - 若$\beta \Rightarrow^* \varepsilon$，那么$\mathrm{FIRST}(\alpha) \cap \mathrm{FOLLOW}(A) = \varnothing$

❑假设$\mathrm{FIRST}(\alpha) \cap \mathrm{FOLLOW}(A) = \{a\}$

$a \in \mathrm{FIRST}(\alpha)$: $A \Rightarrow^* a\alpha'$

$a \in \mathrm{FOLLOW}(A)$: $B \Rightarrow^* \ldots A\, a \ldots$

由于$\beta \Rightarrow^* \varepsilon$，所以遇到$a$时，无法判断用哪一个产生式

- 可以用$A \to \alpha$来对A进行展开
- 亦可以用$A \to \beta$和$\beta \Rightarrow^* \varepsilon$最后把$A$消掉

LL(1)文法

- **LL(1)文法的定义**

  任何两个产生式$A \to \alpha \mid \beta$都满足下列条件:

  - $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \varnothing$
  - 若$\beta \Rightarrow^* \varepsilon$， 那么$\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \varnothing$

□例如, 考虑下面文法

面临a...时， 第2步推导不知用哪个产生式

$S \to A\ B$

$A \to a\ b \mid \varepsilon$　　$a \in \text{FIRST}(ab) \cap \text{FOLLOW}(A)$

$B \to a\ C$

$C \to \ldots$

LL(1)文法

- **LL(1)文法的定义**

  任何两个产生式$A \to \alpha \mid \beta$都满足下列条件：

  - $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \varnothing$
  - 若$\beta \Rightarrow^* \varepsilon$，那么$\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \varnothing$

- **LL(1)文法有一些明显的性质**
  - 没有公共左因子
  - 不是二义的
  - 不含左递归

表达式文法：无左递归的

- 例 $E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \varepsilon$

$F \rightarrow (E) \mid \mathbf{id}$

$\mathbf{FIRST}(E) = \mathbf{FIRST}(T) = \mathbf{FIRST}(F) = \{ (, \mathbf{id} \}$

$\mathbf{FIRST}(E') = \{+, \varepsilon\}$

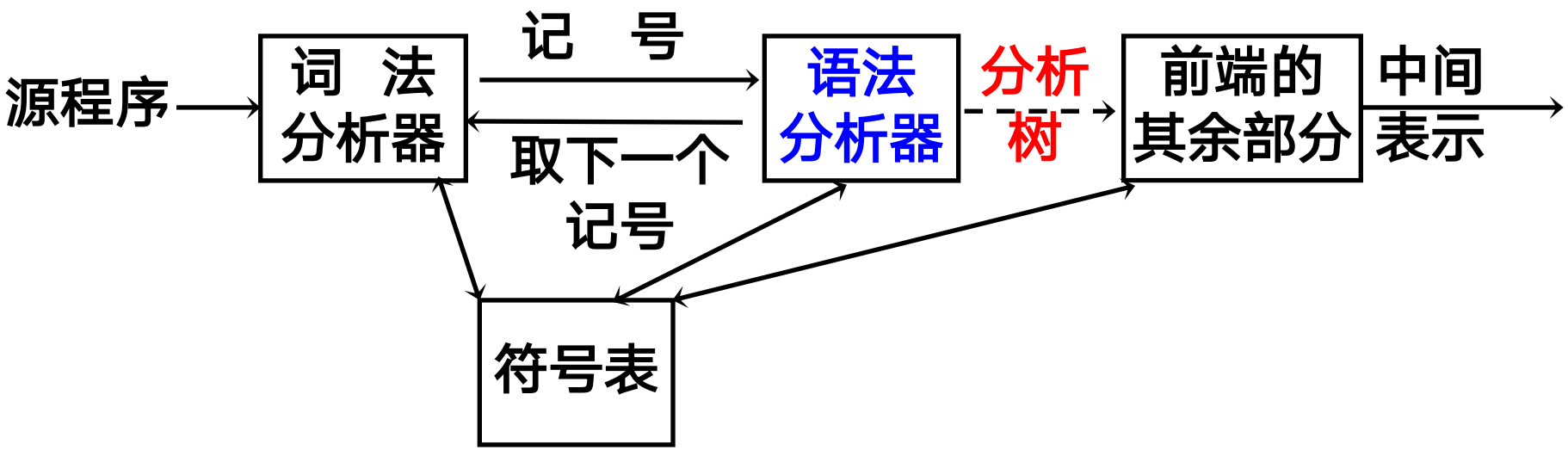
$\mathbf{FRIST}(T') = \{*, \varepsilon\}$

$\mathbf{FOLLOW}(E) = \mathbf{FOLLOW}(E') = \{), \$\}$

$\mathbf{FOLLOW}(T) = \mathbf{FOLLOW}(T') = \{+,), \$\}$

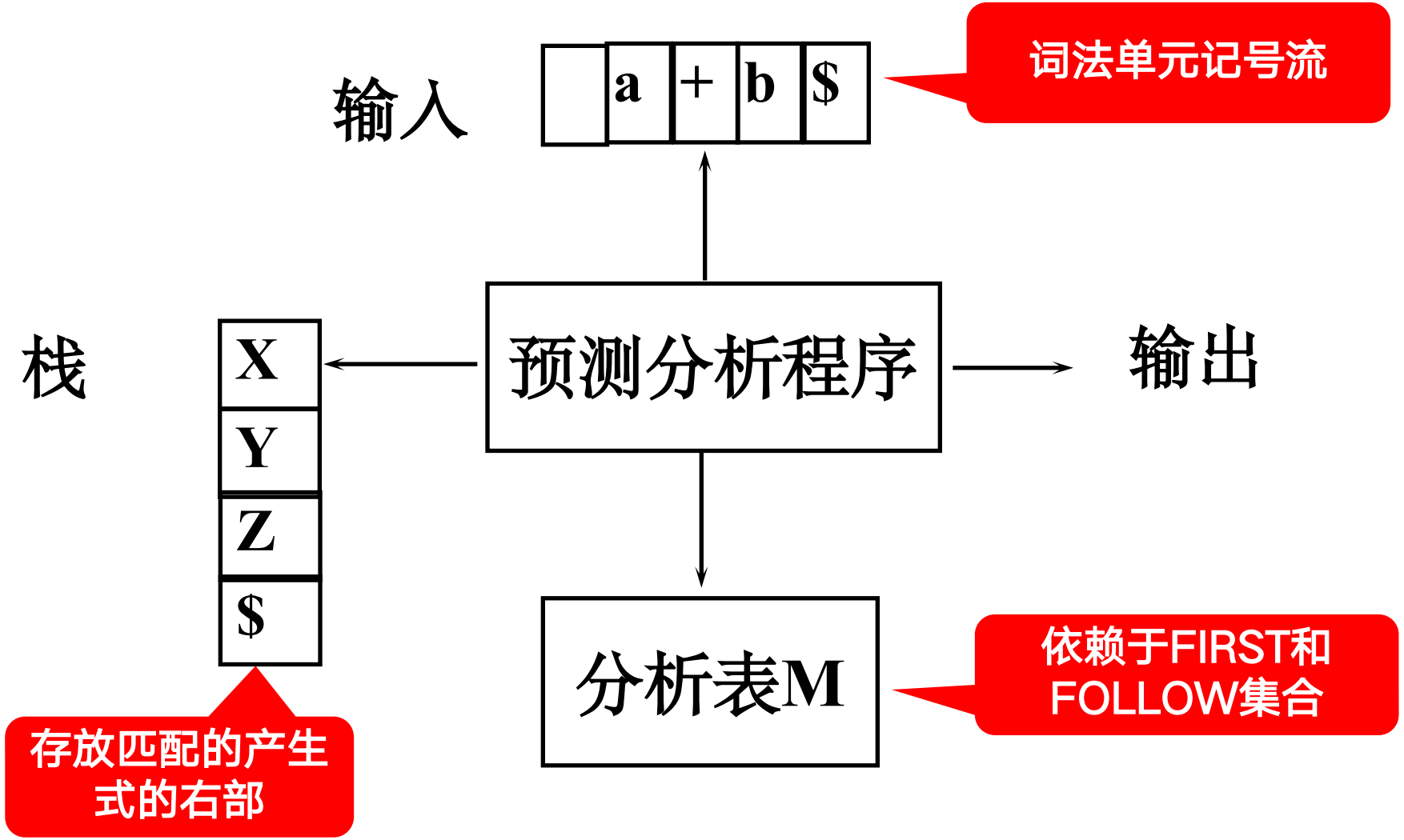
$\mathbf{FOLLOW}(F) = \{+, *,), \$\}$

本节提纲



- 自顶向下分析方法
  - LL(1)文法
  - 非递归预测分析方法

非递归的预测分析


输入
a
+
b
$
词法单元记号流
栈
X
Y
Z
$
预测分析程序
输出
分析表M
依赖于FIRST和FOLLOW集合
存放匹配的产生式的右部

预测分析表M的构造

- 行：非终结符；列：终结符 或$ ；单元：产生式

非终结符	输 入 符 号					
	id	+	*	(	)	$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow$id			$F \rightarrow (E)$		

预测分析表M的构造

- **对文法的每个产生式$A \to \alpha$，执行(1)和(2)**
  - （1）对FIRST(α)的每个终结符a，把$A \to \alpha$加入$M[A, a]$
  - （2）如果ε在FIRST(α)中，对FOLLOW(A)的每个终结符b（包括\$），把$A \to \alpha$加入$M[A, b]$

***M*中其它没有定义的条目都是error**

预测分析举例

预测分析器接受输入id * id + id的前一部分动作

栈	输　入	输　出
$$E$	id * id + id$	

预测分析举例

预测分析器接受输入id * id + id的前一部分动作

栈	输　入	输　出
$$E$	id * id + id$	
$$E'T$	id * id + id$	$E \rightarrow TE'$

预测分析举例

预测分析器接受输入id * id + id的前一部分动作

栈	输　入	输　出
$E	id * id + id$	
$E 'T	id * id + id$	$E \rightarrow TE'$
$E 'T 'F	id * id + id$	$T \rightarrow FT'$

预测分析举例

预测分析器接受输入id * id + id的前一部分动作

栈	输　入	输　出
$$E$	id * id + id$	
$$E'T$	id * id + id$	$E \rightarrow TE'$
$$E'T'F$	id * id + id$	$T \rightarrow FT'$
$$E'T'$ id	id * id + id$	$F \rightarrow$ id

预测分析举例

预测分析器接受输入id * id + id的前一部分动作

栈	输　入	输　出
$$E$	id * id + id$	
$$E'T$	id * id + id$	$E \rightarrow TE'$
$$E'T'F$	id * id + id$	$T \rightarrow FT'$
$$E'T'$ id	id * id + id$	$F \rightarrow$ id
$$E'T'$	* id + id$	匹配id

预测分析举例

预测分析器接受输入id * id + id的前一部分动作

栈	输　入	输　出
$$E$	id * id + id$	
$$E'T$	id * id + id$	$E \rightarrow TE'$
$$E'T'F$	id * id + id$	$T \rightarrow FT'$
$$E'T'$ id	id * id + id$	$F \rightarrow$ id
$$E'T'$	* id + id$	
$$E'T'F$ *	* id + id$	$T' \rightarrow *FT'$

预测分析举例

预测分析器接受输入id * id + id的前一部分动作

栈	输　入	输　出
$$E$	id * id + id$	
$$E'T$	id * id + id$	$E \rightarrow TE'$
$$E'T'F$	id * id + id$	$T \rightarrow FT'$
$$E'T'$ id	id * id + id$	$F \rightarrow$ id
$$E'T'$	* id + id$	
$$E'T'F$ *	* id + id$	$T' \rightarrow *FT'$
$$E'T'F$	id + id$	

预测分析举例

预测分析器接受输入id * id + id的前一部分动作

栈	输　入	输　出
$$E$	id * id + id$	
$$E\,'T$	id * id + id$	$E \rightarrow TE'$
$$E\,'T\,'F$	id * id + id$	$T \rightarrow FT'$
$$E\,'T\,'$ id	id * id + id$	$F \rightarrow$ id
$$E\,'T\,'$	* id + id$	
$$E\,'T\,'F$ *	* id + id$	$T' \rightarrow *FT'$
$$E\,'T\,'F$	id + id$	
$$E\,'T\,'$ id	id + id$	$F \rightarrow$ id

预测分析举例

预测分析器接受输入id * id + id的前一部分动作

栈	输　入	输　出
$E 'T ' id	id + id$	$F \rightarrow$ id
$E 'T '	+ id$	匹配id
$E '	+ id$	$T ' \rightarrow \varepsilon$
$E 'T+	+ id$	$E ' \rightarrow +TE '$
$E 'T	id$	匹配+
$E 'T 'F	id$	$T \rightarrow FT '$
$E 'T 'id	id$	$F \rightarrow$ id
$E 'T '	$	匹配id

预测分析举例

预测分析器接受输入id * id + id的所有动作

栈	输　入	输　出
$E'T'	$	$T' \rightarrow \varepsilon$
$E'	$	$E' \rightarrow \varepsilon$
$	$	Finished

多重定义

例： *stmt* → if *expr* then *stmt* *e_part* | other

e_part → else *stmt*| ε　　*expr* → *b*

非终结符	输入符号			
	other	*b*	else	. . .
stmt	*stmt* → other			
e_part			*e_part*→ else *stmt* *e_part* → ε	
expr		*expr* → *b*		

多重定义条目意味着文法左递归或者是二义的

多重定义的消除

例：删去*e_part* → ε，这正好满足else和近的then配对

LL(1)文法：预测分析表无多重定义的条目

非终结符	输入符号			
	other	*b*	else	. . .
stmt	*stmt* → other			
e_part			*e_part*→ else *stmt* ~~*e_part* → ε~~	
expr		*expr* → *b*		



提取左公因子



- 同一个非终结符的多个产生式**以相同的符号开头**，分析器无法确定应该选择哪个产生式
- 导致回溯（在递归下降中）或直接违反LL(1)文法规则

提取左公因子

stmt -> ‘if’ expr ‘then’ stmt

| ‘if’ expr ‘then’ stmt ‘else’ stmt

| other（代表其他语句）

前两个产生式就具有左公因子 **‘if’ expr ‘then’ stmt**

提取左公因子就是一种文法变换技术，它的目的是**消除这种选择上的不确定性**，将选择推迟到已经看到足够多的输入符号之后。

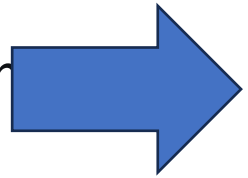
提取左公因子

- 提取左公因子步骤
  - 识别左公因子
  - 将公共前缀提出，创建新的产生式 A->ab，其中a是公共前缀
  - 将原产生式剩余不同部分，作为新的非终结符产生式

stmt -> ‘if’ expr ‘then’ stmt

| ‘if’ expr ‘then’ stmt ‘else’ stm

| other（代表其他语句）



stmt -> ‘if’ expr ‘then’ stmt stmt_suffix

| other

stmt_suffix -> ‘else’ stmt

| ε

一起努力
打造国产基础软硬件体系！

徐 伟

国家高性能计算中心(合肥)、信息与计算机国家级实验教学示范中心

计算机科学与技术学院

2026年09月24日